# Steganografi Çalışma Özeti

LSB ve zayıflıkları, JPEG steganografisi (JSteg → F3 → F4 → F5), matrix embedding / Hamming ve JPEG'e dayanıklı yöntemler. Sadece yüklediğin 4 makaleye dayanıyor; makale dışı bilgiyi açıkça işaretledim.

**Kaynaklar (metinde köşeli parantezle)**


- [1] N. Subramanian, O. Elharrouss, S. Al-Maadeed, A. Bouridane, "Image Steganography: A Review of the Recent Advances," IEEE Access, vol. 9, pp. 23409–23423, 2021.
- [2] J. Fridrich, T. Pevný, J. Kodovský, "Statistically Undetectable JPEG Steganography: Dead Ends, Challenges, and Opportunities," ACM MM&Sec '07, Dallas, 2007.
- [3] Z. Bao, X. Luo, Y. Zhang, C. Yang, F. Liu, "A Robust Image Steganography on Resisting JPEG Compression with No Side Information," IETE Technical Review, vol. 35, no. S1, pp. 4–13, 2018.
- [4] L. Fan, T. Gao, Q. Yang, Y. Cao, "An extended matrix encoding algorithm for steganography of high embedding efficiency," Computers and Electrical Engineering, vol. 37, pp. 973–981, 2011.


## 1. Steganografi nedir, kriptografiden farkı

Steganografi, gizli mesajın **varlığını** saklama sanatıdır: mesaj masum görünen bir kapsayıcının (cover) içine gömülür. [4]'e göre kavramın klasik modeli Simmons'ın 1983 "Prisoners' Problem"ıdır: iki mahkûm, gardiyanı şüphelendirmeden haberleşmek zorundadır.

| | Kriptografi | Steganografi |
|---|---|---|
| Gizlenen şey | Mesajın **içeriği** (okunamaz hale gelir) | Mesajın **varlığı** |
| Çıktı | Şifreli metin, açıkça görünür; bu da şüphe ve inceleme çekebilir [1] | Masum görünen cover/stego nesnesi |
| "Kırılma" ölçütü | İçerik çözülürse | Sadece varlığı tespit edilirse bile kırılmış sayılır [2], [4] |
| Karşı bilim | Kriptanaliz | Steganaliz (stego mu normal mi, ayrıca konum/içerik) [1] |

**Watermarking ile fark:** [1]'e göre steganografi gizliliğe (secrecy), watermarking dayanıklılığa (robustness) vurgu yapar. [3] da robust watermark algoritmalarının steganaliz direncini hedeflemediğini, bu yüzden doğrudan steganografi olarak kullanılamayacağını söyler.

**Steganografik güvenlik (Cachin):** Cover dağılımı Pc, stego dağılımı Ps ise KL mesafesi D(Pc‖Ps) ölçü alınır; D < ε olan sistemlere ε-secure denir, D = 0 ise kusursuz güvenlidir [2]. Pratikte gerçek dağılım bilinmediği için [2] bunun yerine bir sınıflandırıcıyla (SVM) ölçer: P = (P_FA + P_MD) / 2, yani en iyi sınıflandırıcının asgari ortalama hata olasılığı. P = %50 ise tespit yazı-turadan farksızdır.

## 2. Genel süreç: embedding ve extraction

[2]'deki formel tanım: cover x ∈ C, gizli mesaj m ∈ M, gizli anahtar k ∈ K.

- **Embedding:** `y = Emb(x, m, k)` → stego nesnesi y
- **Extraction:** `Ext(y, k) = m` → doğru anahtarla mesajı geri alma

[1]'in Şekil 1'i aynı akışı gösterir: gönderici cover + gizli bilgiyi embedding algoritmasına verir, alıcı stego görüntüden extraction algoritmasıyla mesajı çıkarır; steganaliz ise araya giren gözlemcinin tespit işlemidir.

**Tipik adımlar** (makalelerdeki akışlardan derlendi: [2], [3], [4]):

1. Cover seçimi (ör. gri seviye JPEG, QF 70 [2]).
2. Mesajı bit dizisine çevirme. İstenirse hata düzeltme kodu eklenir (ör. Reed-Solomon [3]).
3. **Gömme alanı:** piksel LSB'leri (spatial) ya da kuantize DCT katsayıları (transform).
4. **Konum seçimi:** sıralı, anahtarla sözde-rastgele yol veya içeriğe duyarlı (content-adaptive) seçim kuralı [2].
5. **Gömme işlemi:** bit değiştirme (LSB flip, ±1 vb.), çoğu zaman bir kodlama ile (matrix embedding, wet paper codes, STC) birlikte.
6. Stego nesnesini gönderme.
7. Alıcıda aynı anahtar/kodla extraction.

### Ölçütler

- **Kapasite:** log2|M| bit [2]. Birimleri: bpp = L/(H·W·C) [1]; JPEG'de bpac (bits per non-zero AC coefficient) [2].
- **Embedding efficiency:** e = log2|M| / E{d(x,y)}, yani birim bozulma başına gömülen bit [2]. [4]'te: değişiklik başına ortalama gömülen bit.
- **Görsel kalite:** MSE ve PSNR = 10·log10($R^2$/MSE); 8-bit için R = 255 [1].
- **Güvenlik:** steganaliz doğruluğu veya hata olasılığı P [1], [2].

Önemli: bu ölçütler birbirine karşı çalışır; kapasiteyi artırmak genelde gizliliği düşürür, gürültüye çok dayanıklı yöntemler ise kapasite ya da gizlilikten ödün verir [1].

## 3. Spatial domain vs transform domain

| | Spatial domain | Transform domain (JPEG/DCT) |
|---|---|---|
| Nereye gömülür | Piksel değerlerinin LSB'si | Kuantize DCT katsayıları |
| Örnek | LSB, PVD [1] | JSteg, F3/F4/F5, Steghide, MBS [2], [4] |
| Zayıflık | Piksel değişimine ve görsel saldırıya açık [4] | Katsayı histogramı ve blok bağımlılıkları iz bırakabilir [2] |
| Avantaj | Basit, yüksek kapasite | Daha az dikkat çekici bölgeler; JPEG en yaygın format olduğu için en çok çalışılan alan [2], [4] |

**JPEG hattı** [3]: renk uzayı dönüşümü → alt örnekleme → 8×8 bloklara bölme → her blokta DCT → katsayıları kalite faktörüne (QF) göre tam sayıya kuantize etme → kodlama. JPEG adaptif steganografi algoritmaları gömmeyi bu kuantize edilmiş DCT katsayıları üzerinde yapar [3]. Katsayılar DC (blok ortalaması) ve 63 AC katsayısıdır; F5 sadece sıfır olmayan AC katsayılarına gömer [2].

## 4. LSB steganografisi ve zayıflıkları

**Nasıl çalışır?** Cover görüntünün piksellerinin en düşük anlamlı bitleri, gizli mesajın bitleriyle değiştirilir; varsayım, birkaç piksel değerini oynatmanın gözle görülmeyeceğidir [1]. LSB düzlemi rastgele ve gürültülü göründüğü için tercih edilir [4].

**Matematik:** Mesaj bitleri ve cover bitleri 0–1 düzgün dağılımlıysa gömülecek bit yarı olasılıkla zaten aynıdır. Yani her pozisyon ortalama %50 olasılıkla değişir; embedding efficiency 2 bit/değişiklik olur. Görüntü tamamen doldurulursa cover verisinin %50'si değişir ve bu istatistiksel steganaliz için büyük risktir [4].

**Zayıflıklar** (hepsi makalelerden):

1. **Aşırı yükleme görünür iz bırakır.** Cover çok doldurulursa değişiklikler gözle fark edilir [1].
2. **Spatial domain yöntemleri piksel değişimine ve görsel saldırıya karşı savunmasızdır** [4].

3. **Gömme operasyonunun kendisi tespit edilebilir.** [2], LSB çevirmenin kendisini yüksek tespit edilebilirliğin sebebi olarak gösterir (Ker'in yapısal analizine atıfla).
4. **Değer çiftleri (POV) problemi.** LSB değişimi ardışık iki değerin (2k, 2k+1) frekanslarını eşitlemeye iter. JSteg'de bu histogramda belirgin iz bırakır ve Westfeld-Pfitzmann'ın $\chi^2$ testiyle yakalanır [4].
5. **Çok düşük oranda bile tespit.** JSteg tarzı gömme, 0.05 bit/kullanılabilir katsayı gibi çok düşük oranlarda bile genelleştirilmiş category attack ile güvenilir biçimde bulunabilir [2].
6. **Kayıplı kanalda mesaj kaybolur.** JPEG sıkıştırma gibi kayıplı işlemler geleneksel steganografilerle gizlenmiş mesajı çoğu zaman bozar [3].
7. **Düşük güvenlik/kapasite dengesi.** [1]'e göre geleneksel yöntemler (LSB ve türevleri) derin öğrenme yöntemlerine göre daha az güvenlidir ve kapasiteleri sınırlıdır; fazla piksel kullanmak bozulmaya yol açar.

**LSB varyantları** ([1]'in geleneksel yöntemler bölümü):

- **Huffman + LSB:** mesaj önce Huffman ile kodlanıp sonra LSB ile gömülür.
- **k-LSB:** en düşük k bit mesajla değiştirilir.
- **RGB LSB:** üç kanala bit-slicing ile 2:2:4 oranında dağıtım.
- **LSB + PVD (Pixel Value Differencing):** ardışık piksellerin farkına göre gömülecek bit sayısı belirlenir; 8 bit için ilk iki bitte LSB, kalan altı bitte PVD.
- **Dinamik anahtarlı LSB:** [1]'de Patel & Meena'nın çalışması olarak geçer: sözde-rastgele sayı üretici ile gömülecek pikseller seçilir, anahtar her seferinde döndürme ile şifrelenir. Bu, "sıralı gömme" yerine "anahtara bağlı rastgele konum" fikridir. JSteg'in sıralı yoldan sözde-rastgele yola geçmesiyle ([2]) ve Outguess'in PRNG ile saçmasıyla ([4]) aynı mantıktır.

Not: "Dynamic LSB" terimi bu makalelerde tek bir standart algoritma olarak tanımlanmıyor; sunumda yukarıdaki anlamıyla ve [1]'e atıfla anlat.

## 5. JPEG steganografisinin evrimi: JSteg → Outguess → F3 → F4 → F5

Akış bir kedi-fare oyunu: her yöntem bir öncekinin yakalandığı noktayı düzeltmeye çalışır.

| Yöntem | Fikir | Neden yakalandı / sorunu |
|---|---|---|
| **JSteg** | Kuantize DCT katsayılarının LSB'si; 0 ve 1 değerli katsayılar atlanır; yol sıralı (sonra sözde-rastgele) [2] | Histogramda POV izi; $\chi^2$ testi, histogram saldırısı [2], [4] |

| Yöntem | Fikir | Neden yakalandı / sorunu |
|---|---|---|
| **Outguess 0.1** | Gömme konumlarını PRNG ile tüm DCT alanına dağıtır [4] | Rastgele dağılsa da istatistiksel iz kalır [4] |
| **Outguess 0.2** | Gömme sonrası istatistik düzeltmesi (statistical restoration) [4] | Blok sınırı süreksizliği ile gömülen veri uzunluğu kestirilebildi [4]. Düzeltme adımı tespit edilebilirliği artırabilir [2] |
| **F3** | Mesaj biti ile katsayının LSB'si aynıysa değişiklik yok; değilse **mutlak değer 1 azaltılır** [4] | POV problemini çözer ama çift katsayılar fazla olur, histogram doğal görünmez [4] |
| **F4** | Çift-negatif ve tek-pozitif katsayı = 1, tek-negatif ve çift-pozitif = 0 sayılır; eşleşmezse mutlak değer 1 azaltılır [4] | Histogram temiz bir görüntüye benzer [4] |
| **F5** | Mutlak değeri azaltma + **matrix embedding** + permutative straddling [2], [4] | Shrinkage (aşağıda); sonraki çalışmalar yine de tespit etti |

Not: F1/F2 bu makalelerde geçmiyor, anlatım F3'ten başlıyor [4]. Hoca "F1/F2" dediyse neyi kastettiğini teyit etmek iyi olur.

## F5'in üç ana unsuru

1. **Değiştirme operasyonu:** katsayının mutlak değeri her zaman 1 azaltılır, LSB "çevrilmez". [2]'ye göre histogramdaki bariz izler böylece ortadan kalkar; çünkü LSB çevirmenin kendisi yüksek tespit edilebilirliğe sebep olur.
2. **Matrix embedding (syndrome coding):** Hamming kodları ile daha az değişiklikle daha çok bit gömülür (bölüm 6).
3. **Permutative straddling:** gizli mesajı tüm cover üzerine dağıtır; güvenliği artırır [4].

**Shrinkage:** F5 sadece sıfır olmayan AC katsayılarına gömer. Bir katsayı 1 ya da −1 iken azaltılınca 0 olursa (shrinkage), alıcı sıfırları atlayarak okuduğu için o bit bir sonraki katsayıya tekrar gömülür. Bu hem verimliliği düşürür hem histogramı etkiler. F5, bunu hafifletmek için ±1 katsayıların yaklaşık %50'sini atlar [2].

**F5 türevleri ([2]'nin deneyleri):**

- **−F5:** shrinkage'ı kaldırmak için değeri azaltmak yerine artırır. Beklenenin aksine orijinal F5'ten kötü çıkar; bu F5 tasarımcısının seçimini doğrular.
- **nsF5 (no-shrinkage F5):** shrinkage'ı wet paper codes ile çözer. Yan bilgi kullanmayan yöntemler arasında en iyisidir [2].

# 6. Matrix embedding ve Hamming

**Fikir:** Cover'ın LSB vektörü x (n bit), mesaj m (k bit), H bir k×n ikili matris. Sentez: `Hx` hesaplanır; `Hx = m` ise hiç değişiklik gerekmez. Değilse `Hx − m` farkı H'nin tam bir sütunudur (j. sütun); x'in j. bitini çevirmek yeter. Alıcı `m = Hy` ile mesajı sendrom olarak okur [2].

**n = 7, k = 3 örneği** ([2]'deki H matrisi):

```
H = 1 0 0 1 1 0 1
    0 1 0 1 0 1 1
    0 0 1 0 1 1 1
```

H'nin sütunları 3 bitlik tüm sıfırdan farklı vektörlerdir (1..7'nin ikili yazılışı).

Örnek: cover LSB'leri x = 1011001, mesaj m = 011.

- `Hx = (1,0,0)`, bu `m = (0,1,1)`'e eşit değil.
- `Hx ⊕ m = (1,1,1)` = H'nin 7. sütunu → x'in **7. bitini** çevir.
- y = 1011000 → `Hy = (0,1,1) = m` ✓. 7 pikselde 3 bit, tek değişiklikle.

**Hash fonksiyonu (Fan et al. gösterimi) [4]:** aynı şey şöyle de yazılır: `f(a) = ⊕ (a_i · i)`, yani değeri 1 olan bitlerin indekslerinin XOR'u. `y = w ⊕ f(a)` bulunur ve a'nın y. biti çevrilir (y = 0 ise dokunulmaz). Küçük örnek: (n,k) = (3,2), a = 101, w = 11. f(a) = 1 ⊕ 3 = 01 ⊕ 11 = 10 (=2); y = 11 ⊕ 10 = 01 (=1); 1. biti çevir: a′ = 001; f(a′) = 3 = w ✓. Alıcı sadece f(a′)'yi hesaplar.

**Verimlilik formülleri:**

- [2]: p bit mesaj, 2^p − 1 pikselde; en fazla 1 değişiklik; ortalama 1 − 2^−p değişiklik. Göreli yük α_p = p/(2^p − 1), verimlilik e_p = p/(1 − 2^−p) (bit/sıfır olmayan AC katsayısı).
- [4]: değişiklik yoğunluğu D = 1/(n+1), gömme oranı R = k/n, verimlilik E = R/D = ((n+1)/n)·k; F5'te n = 2^k − 1.

| (n,k) | D (%) | R (%) | E (bit/değişiklik) |
|---|---|---|---|
| (1,1) | 50 | 100 | 2 |
| (3,2) | 25 | 66.67 | 2.67 |
| (7,3) | 12.5 | 42.86 | 3.43 |
| (15,4) | 6.25 | 26.67 | 4.27 |
| (31,5) | 3.13 | 16.13 | 5.16 |

| (n,k) | D (%) | R (%) | E (bit/değişiklik) |
|---|---|---|---|
| (63,6) | 1.56 | 9.52 | 6.09 |
| (255,8) | 0.39 | 3.14 | 8.03 |

(Tablo [4]'ün Table 1'inden.) Yorum: k büyüdükçe verimlilik artar ama oran düşer: "verimliliğin bedeli düşük orandır" [4].

**Hamming (7,4) bağlantısı** (makale dışı, kodlama teorisi bilgisi): Yukarıdaki H, Hamming(7,4) kodunun parite-kontrol matrisidir (7 bitlik kod kelimesi: 4 veri + 3 parite biti). [2] bunu "binary Hamming codes" diye anar; "(7,4)" etiketi makalelerde geçmiyor. Sunumda bu bağı kurarsan kodlama teorisi kitabından (ör. standart bir Hamming kodu referansı) atıf eklemeni öneririm. Sezgi: Hamming kodu tek hata düzeltir. Burada "hata" yerine "tek bit değişikliği" ile her sendromu üretebiliriz; sendrom = mesaj.

**Wet paper codes ve STC:**

- **Wet paper codes** [2]: gönderici, alıcının bilmediği yan bilgiyi kullanabilir (ör. hangi katsayıların sıfırlandığı). Denklem `Dy = m`, D paylaşılan rastgele ikili p×n matristir; x_i = y_i koşulu "wet" (değiştirilemez) pozisyonlarda sağlanır. Sürekli bir kod ailesi sunar, Hamming'in kesikli seçeneklerinden daha esnektir. F5'in shrinkage problemini bu yüzden çözer (nsF5).
- **Syndrome-Trellis Codes (STC)** [3]: modern adaptif steganografinin standardı: "gömme maliyet fonksiyonu + STC". `msg = y·H^t`; H, köşegen boyunca dizilmiş küçük alt matrislerden oluşur (yükseklik h, genişlik w), kod Viterbi mantığıyla en düşük maliyetli y'yi bulur. h genelde 8–12 aralığındadır [3].

## 7. Fan et al.: genişletilmiş matrix encoding [4]

**Problem:** Standart matrix encoding'de n = 2^k − 1 zorunludur; k arttıkça hücre uzunluğu üstel büyür, oran çöker. Genişletilmiş algoritma, k'yı n sabitken artırmaya çalışır.

**Fikir (layer extension):** Kodlama üçlüsü (d_max, n, k) → dörtlü (d_max, n, k, L). Hash çıktısı indeksi 2^l ile çarpılarak ek katmanlar açılır. Hücre n bitlik olsa da yalnızca belirli "genişletilmiş kodlar" gömülebilir (ör. 3 bitlik hücrede 1-layer ile 000, 010, 100, 110). Gönderici en uzun gizli kod w'den başlayıp mod(w, 2^l) = 0 sağlayan en büyük katmanı bulur; yoksa standart matrix encoding'e döner.

**Formüller:** hash `f(a) = ⊕ a_i·(i·2^l_crt)`; `y = (w ⊕ f(a)) / 2^l_crt`. Alıcının hangi katmanda olduğunu bilmesi için hücreye m = ⌈log2(L+1)⌉ bitlik **sembol** s eklenir (stego hücre n + m bit).

**Örnek (1,7,3,2):** a = 1101010, w = 11001. mod(11001, $2^2$) ≠ 0 → daha kısa w = 1100 dene; mod(1100, $2^1$) = 0 → katman l = 1. Sembol uzunluğu m = ⌈log2 3⌉ = 2, s = 01. f(a) = 0010, y = (1100 ⊕ 0010)/2 = 7 → 7. biti çevir → a′ = 1101011; stego hücre c = 110101101. Alıcı s = 01 → l = 1, f(a′) ile w = 1100′ü bulur.

**Sonuçlar ve sınırlar:**

- Gömme oranı teorik olarak %100′ü aşabilir [4].
- k/n büyükken verimlilik F5′ten yüksek; k/n küçükken (ör. L = 7 için k/n < 3/7) sembol bitleri yüzünden F5′ten düşük [4].
- Sembol ayarı için ortalama m/2 bit değişiklik gerekir; bunlar mesaj taşımaz [4].
- Yazarlar, deney mesajı olarak ardışık siyah piksel (00000…) içeren ikili logo kullandı; rastgele mesajlarda genişletme ancak 0.5 olasılıkla mümkün [4]. Yani kazanç mesajın yapısına bağlı. Bu, sunumda dürüstçe söylenecek bir zayıflık.
- Deneyde Lena 256×256, 64×64 logo; PSNR 70.33 [4].

## 8. JPEG steganografisinin tespit edilebilirliği: [2]'nin bulguları

**Düzen:** 6006 ham görüntü, gri seviye JPEG, QF 70 (PQ için 85/70 çift sıkıştırma). Steganaliz: 274 boyutlu öznitelik (193 genişletilmiş DCT + 81 Markov) ile Gauss çekirdekli SVM. Metrik: hata olasılığı P (yüksek = gizli, 50 = rastgele tahmin). Yük birimi bpac.

| Yöntem | P @ 0.05 bpac | P @ 0.20 bpac |
|---|---|---|
| Steghide | %2.58 | %0.26 |
| MBS | %4.61 | %0.36 |
| JP Hide&Seek | %4.01 | %2.11 |
| PQ | %6.73 | %3.19 |
| –F5 | %7.37 | %0.12 |
| F5 | %12.94 | %0.80 |
| nsF5 | %28.90 | %3.21 |
| MM2 | %38.26 | %0.82 |
| PQe | %37.80 | %16.15 |
| PQt | %40.06 | %15.27 |
| MM3 | %40.86 | %1.24 |

**Çıkarımlar** [2]:

- Mevcut en iyi kör steganalize göre ortalama güvenli kapasite ≈ **0.05 bpac** (gri, QF 70).
- Steghide, MBS ve JPHS en kolay tespit edilenlerdir; **istatistiği koruyan** yöntemler (Steghide, MBS) buna rağmen heuristik tasarlanmış, matrix embedding kullanan yöntemlerden (F5, nsF5) daha kötü çıkar.
- **Matrix embedding** değişiklik sayısını seyrelterek güvenliği belirgin artırır; önemi yük azaldıkça artar.
- Yan bilgi gerektirmeyen en iyi yöntem nsF5; yan bilgi (sıkıştırılmamış cover) kullananlar PQt ve MM3.
- İçerik-adaptif seçim (doku ve enerji tabanlı PQt/PQe) güvenliği artırır, ancak seçim kuralı saldırgana bilgi sızdırabilir.
- **JPEG sıkıştırıcısı** da sonucu etkiler: cover'ı farklı sıkıştırıcıyla üretmek nsF5'in tespit edilebilirliğini kısa mesajlarda %15'e kadar yapay biçimde artırabilir.

Not: Bu sonuçlar 2007'ye ait; daha yeni adaptif yöntemler (J-UNIWARD vb.) [3]'te anılıyor.

## 9. JPEG sıkıştırmaya dayanıklı steganografi [3]

**Motivasyon:** Sosyal medya (Facebook, WeChat, Instagram) görüntüleri bant genişliği vb. nedenlerle JPEG ile yeniden sıkıştırır. Bu kayıplı işlem geleneksel yöntemlerin mesajını bozar. Çözüm arayışı "robust steganography" olur: kayıplı kanaldan geçsin ama yine de steganalize dayansın.

**Zincir:** adaptif steganografi = **gömme maliyet fonksiyonu + STC** (ör. J-UNIWARD, UED). Bunlar kayıpsız kanalda iyidir, kayıplı kanalda çöker. DCRAS ve FRAS dayanıklıdır ama saldırı sıkıştırmasının QF'sini bilmeyi gerektirir (yan bilgi).

**Önerilen yöntem (QF bilgisi gerekmeden):**

1. **Hata yayılımı analizi:** STC'nin H matrisinin alt matris yüksekliği h ve genişliği w ne kadar büyükse çıkarılan mesajdaki hata olasılığı o kadar yüksektir; bir stego bitindeki hata h mesaj bitini etkiler.
2. **Robust sanal cover:** Her 8×8 bloktaki DCT katsayısı $D_k(i)$, komşu üç bloğun aynı konumdaki katsayılarının ortalamasıyla karşılaştırılır; büyükse 1, küçük/eşitse 0. Mesaj bu sanal bit dizisine gömülür. İlişkisel olduğu için sıkıştırmada daha kararlıdır.
3. **Maliyet fonksiyonu:** J-UNIWARD maliyeti sanal cover'a uyarlanır.
4. **Hata yayılımını azaltma:** w'yi küçültmek için gömme oranı düşürülür; düşük frekanslı konumlar seçilir. **Reed-Solomon (6,15)** ile hata düzeltme uygulanır.

**Sonuç:** BOSSbase 1.01 (10.000 görüntü), QF 75 cover, saldırı QF 65–85: çıkarma hata oranı ≈ %0.3–3 (karşılaştırılan robust watermark yöntemi %3–10). Steganaliz (ccPEV, DCTR) karşısında DCRAS'tan en fazla %7 daha kötü. Yazarlara göre sebep, STC'nin tüm cover

yerine bir kısmında çalışması. Bu bölüm hocanın "görüntü bozulması ile steganografi" konusuyla doğrudan ilgili.

## 10. Derin öğrenme tabanlı steganografi (kısa harita) [1]

[1] yöntemleri üç sınıfa ayırır:

- **Geleneksel:** LSB, PVD, DCT/DWT, EMD vb. Kapasite düşük, güvenlik daha az.
- **CNN tabanlı:** encoder-decoder (U-Net, VGG tabanlı). Cover ve gizli görüntü birleştirilip stego üretilir, decoder gizli görüntüyü çıkarır. Kapasite çok yüksektir (neredeyse 1 bpp mertebesi).
- **GAN tabanlı:** generator + discriminator (+ steganalyzer); SGAN, SSGAN, HiDDeN, CycleGAN türevleri, coverless yöntemler. Güvenlik ve PSNR'de en iyi sonuçlar (örn. CycleGAN tabanlı yöntemle 64.7 dB).

Değerlendirme ölçütleri: PSNR/MSE/SSIM, steganaliz doğruluğu, BPP. Veri setleri: BOSSBase (10.000 gri 512×512), ImageNet, CelebA, COCO vb. Açık problemler: ortak benchmark yok, GAN yakınsama ve mode collapse, gerçek zamanlı kullanım, man-in-the-middle ve bozulma saldırılarına dayanıklılık [1].

## 11. Sunumda kullanabileceğin özet tablo: yöntem → fikir → zayıflık

| Yöntem | Alan | Fikir | Zayıflık |
|---|---|---|---|
| LSB | Spatial | Piksel LSB'sini mesajla değiştir | Görsel/istatistiksel saldırı, %50 değişim, kayıplı kanalda ölür |
| Dinamik anahtarlı LSB | Spatial | PRNG ile konum seç | Hâlâ LSB'nin istatistiksel izi |
| PVD+LSB | Spatial | Piksel farkına göre bit sayısı | Dokuya bağlı kapasite, farklı histogram izi |
| JSteg | DCT | DCT LSB, 0/1 hariç | POV, $\chi^2$ ve histogram saldırısı |
| Outguess | DCT | PRNG saçma, istatistik düzeltme | Blokluluk saldırısı, düzeltme izi |
| F3 | DCT | | değer |
| F4 | DCT | İşaretle eşleme + | değer |
| F5 | DCT | | değer |

| Yöntem | Alan | Fikir | Zayıflık |
|---|---|---|---|
| nsF5 | DCT | F5 + wet paper codes | Yan bilgi yok; PQ/MMx kadar güvenli değil |
| Genişletilmiş ME [4] | DCT | Layer extension, oran > %100 | k/n küçükken verimlilik düşer, mesaja bağlı |
| DCRAS / Bao et al. [3] | DCT | Robust sanal cover + STC + RS | Tespit edilebilirlik biraz yüksek |

## 12. Sunuma hazırlık: olası sorular

- **Neden ±1 azaltma, LSB çevirmeden daha iyi?** Çünkü LSB çevirme çift (2k,2k+1) değerlerini dengeler ve iz bırakır; azaltma bu simetrik eşitlemeyi yapmaz [2], [4].
- **Matrix embedding neden güvenliği artırır?** Değişiklik sayısı azalır, yani değişiklik yoğunluğu düşer; embedding efficiency yükselir [2], [4].
- **Shrinkage nedir, neden problem?** Sıfırlanan katsayıyı alıcı göremez; bit tekrar gömülür, verimlilik ve histogram bozulur [2].
- **Kapasite ne kadar?** Güvenli kapasite ≈ 0.05 bpac [2].
- **Sosyal medya/kayıplı kanalda ne olur?** Sıkıştırma mesajı bozar; robust yöntemler sanal cover + STC + hata düzeltme kullanır [3].
- **Daha güncel yaklaşımlar?** CNN/GAN tabanlı yöntemler [1].

## 13. Atıf notları

- Yalnızca bu dokümandaki [1]–[4] kaynaklarına dayanan ifadeler atıflıdır. "Hamming(7,4)" adlandırması ve Hamming kodunun tek hata düzeltme özelliği makale dışı genel kodlama teorisi bilgisidir; sunumda kullanacaksan bir kodlama teorisi kaynağıyla destekle.
- Makalelerin içinde geçen birincil kaynaklara (Westfeld F5, Crandall, Sallee MBS, Filler STC, Fridrich wet paper codes vb.) doğrudan atıf yapmak istersen, ilgili makalenin kaynakça bölümünden künyeyi alabilirsin; ben burada ikincil kaynak olarak [1]–[4] üzerinden gösterdim.